OS - Orange Problem

Name: A U Divyaa Rukshana
SRN: PES2UG24AM002
Section: AIML - 4A

SUBMISSIONS

Screenshots

Phase 1:

1A)

```
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B)

```
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase 2:

2A)

```
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ ./test_tree

Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$
```

2B)

```
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ find .pes/objects -type f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/12/07265c697b6d87477e9364d4d8f8d6f38b63efcce5a66089c3a093e51a308a
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ xxd .pes/objects/12/07265c697b6d87477e9364d4d8f8d6f38b63efcce5a66089c3a0
93e51a308a | head -20
00000000: 626c 6f62 2031 3400 696e 7420 6d61 696e  blob 14.int main
00000010: 2829 207b 7d0a                           () {}.
```

Phase 3:

3A)

```
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ make pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ echo "hello" > file1.txt
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ echo "world" > file2.txt
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ ./pes add file1.txt file2.txt
```

```
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_tree.c
  untracked:  pes.c
  untracked:  index.h
  untracked:  object.c
  untracked:  tree.c
  untracked:  test_sequence.sh
  untracked:  index.c
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  commit.h
  untracked:  Makefile
  untracked:  pes.h
  untracked:  .gitignore
  untracked:  tree.h
  untracked:  commit.c
```

3B)

```
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776431867 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776431876 6 file2.txt
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$
```

Phase 4:

4A)

```
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ ./pes log
commit b8b1bdd3a0123218a66cd4bbd8499b4f28346f6fbadd33185993d3ee85ec1609
Author: PES User <pes@localhost>
Date:   1776434530

    Add farewell

commit e847dd0b3a454020acf6606e1a807a23cff6cc37709f9da6e6d953c2abe74a0f
Author: PES User <pes@localhost>
Date:   1776434501

    Add worldell

commit 46f929c54b43958084f2a4824093feb4509047c1c761cca418ed318c5c53f345
Author: PES User <pes@localhost>
Date:   1776434477

    Initial commit

commit 440f33e6300197a146171e31061f58eb61f24cea2f89b1bd700603cc9ebaa646
Author: PES User <pes@localhost>
Date:   1776434338

    test committest committ>Q
```

4B)

```
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/37/1aac45c3ba401d13c2ce98c9537b4e53c6049b950bf8a33cbc400712d9f4fa
.pes/objects/44/0f33e6300197a146171e31061f58eb61f24cea2f89b1bd700603cc9ebaa646
.pes/objects/46/f929c54b43958084f2a4824093feb4509047c1c761cca418ed318c5c53f345
.pes/objects/63/2d7ba2be92407b48aa6b0357ea1c5a0b0e082419ca96ce991038a5bee63b0e
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/b8/b1bdd3a0123218a66cd4bbd8499b4f28346f6fbadd33185993d3ee85ec1609
.pes/objects/cd/96c8067a02ae6bf08c852f1563d89918daeda45ce0a15e2a6eb8142278be03
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/e8/47dd0b3a454020acf6606e1a807a23cff6cc37709f9da6e6d953c2abe74a0f
.pes/objects/f2/2497c605be7b1c0ad3b32bcfdd1e526e9703da6ba25654ad12ef5fe05a17fd
.pes/refs/heads/main
```

4C)

```
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ cat .pes/refs/heads/main
b8b1bdd3a0123218a66cd4bbd8499b4f28346f6fbadd33185993d3ee85ec1609
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Final:

```
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed as 73272ca9644c61d75f34066bf84954efad90e5ef8f956048d816f0e80ce79353
Committed: 73272ca9644c... Initial commit

Log after first commit:
commit 73272ca9644c61d75f34066bf84954efad90e5ef8f956048d816f0e80ce79353
```

```
Log after first commit:
commit 73272ca9644c61d75f34066bf84954efad90e5ef8f956048d816f0e80ce79353
Author: PES User <pes@localhost>
Date:   1776434789

    Initial commit


--- Second Commit ---
Committed as 0b37500f001714113a88d1fc06d1bb91e7abdc869836bd1d200b68b9a4525150
Committed: 0b37500f0017... Update file.txt

--- Third Commit ---
Committed as 7ee307831729d66d466496b3d438f05886dfd846e3270580ad547b639b750d1a
Committed: 7ee307831729... Add farewell

--- Full History ---
commit 7ee307831729d66d466496b3d438f05886dfd846e3270580ad547b639b750d1a
Author: PES User <pes@localhost>
Date:   1776434789

    Add farewell

commit 0b37500f001714113a88d1fc06d1bb91e7abdc869836bd1d200b68b9a4525150
Author: PES User <pes@localhost>
Date:   1776434789

    Update file.txt

commit 73272ca9644c61d75f34066bf84954efad90e5ef8f956048d816f0e80ce79353
```

```
    Update file.txt

commit 73272ca9644c61d75f34066bf84954efad90e5ef8f956048d816f0e80ce79353
Author: PES User <pes@localhost>
Date:   1776434789

    Initial committial commiQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
7ee307831729d66d466496b3d438f05886dfd846e3270580ad547b639b750d1a

--- Object Store ---
Objects created:
10
.pes/objects/0b/37500f001714113a88d1fc06d1bb91e7abdc869836bd1d200b68b9a4525150
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/73/272ca9644c61d75f34066bf84954efad90e5ef8f956048d816f0e80ce79353
.pes/objects/7e/e307831729d66d466496b3d438f05886dfd846e3270580ad547b639b750d1a
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
divyaa@divyaa-vm:~/PES2UG24AM002-pes-vcs$
```

Code Files

1. **object.c**

```
// object.c — Content-addressable object store
//
// Every piece of data (file contents, directory listings, commits) is stored
// as an "object" named by its SHA-256 hash. Objects are stored under
// .pes/objects/XX/YYYYYY... where XX is the first two hex characters of the
// hash (directory sharding).
//
// PROVIDED functions: compute_hash, object_path, object_exists, hash_to_hex,
hex_to_hash
// TODO functions:     object_write, object_read

#include "pes.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <openssl/evp.h>

// ——— PROVIDED ———————————————
```

```
    if (strlen(hex) < HASH_HEX_SIZE) return -1;
    for (int i = 0; i < HASH_SIZE; i++) {
        unsigned int byte;
        if (sscanf(hex + i * 2, "%2x", &byte) != 1) return -1;
        id_out->hash[i] = (uint8_t)byte;
    }
    return 0;
}

void compute_hash(const void *data, size_t len, ObjectID *id_out) {
    unsigned int hash_len;
    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, EVP_sha256(), NULL);
    EVP_DigestUpdate(ctx, data, len);
    EVP_DigestFinal_ex(ctx, id_out->hash, &hash_len);
    EVP_MD_CTX_free(ctx);
}

// Get the filesystem path where an object should be stored.
// Format: .pes/objects/XX/YYYYYYYY...
// The first 2 hex chars form the shard directory; the rest is the filename.
void object_path(const ObjectID *id, char *path_out, size_t path_size) {
    char hex[HASH_HEX_SIZE + 1];
    hash_to_hex(id, hex);
    snprintf(path_out, path_size, "%s/%.2s/%s", OBJECTS_DIR, hex, hex + 2);
}

int object_exists(const ObjectID *id) {
    char path[512];
    object_path(id, path, sizeof(path));
    return access(path, F_OK) == 0;
}

// ─── TODO: Implement these
```

```
// Write an object to the store.
```

```
//
// Object format on disk:
//   "<type> <size>\0<data>"
//   where <type> is "blob", "tree", or "commit"
//   and <size> is the decimal string of the data length
//
// Steps:
//   1. Build the full object: header ("blob 16\0") + data
//   2. Compute SHA-256 hash of the FULL object (header + data)
//   3. Check if object already exists (deduplication) — if so, just return success
//   4. Create shard directory (.pes/objects/XX/) if it doesn't exist
//   5. Write to a temporary file in the same shard directory
//   6. fsync() the temporary file to ensure data reaches disk
//   7. rename() the temp file to the final path (atomic on POSIX)
//   8. Open and fsync() the shard directory to persist the rename
//   9. Store the computed hash in *id_out

// HINTS - Useful syscalls and functions for this phase:
//   - sprintf / snprintf : formatting the header string
//   - compute_hash       : hashing the combined header + data
//   - object_exists      : checking for deduplication
//   - mkdir              : creating the shard directory (use mode 0755)
//   - open, write, close : creating and writing to the temp file
//                          (Use O_CREAT | O_WRONLY | O_TRUNC, mode 0644)
//   - fsync              : flushing the file descriptor to disk
//   - rename             : atomically moving the temp file to the final path
//

//
// Returns 0 on success, -1 on error.
int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out) {
    char header[64];
    const char *type_str;

    if (type == OBJ_BLOB) type_str = "blob";
    else if (type == OBJ_TREE) type_str = "tree";
    else if (type == OBJ_COMMIT) type_str = "commit";
```

```
    else return -1;

    int header_len = snprintf(header, sizeof(header), "%s %zu", type_str, len) + 1;

    size_t total_len = header_len + len;
    char *full = malloc(total_len);
    if (!full) return -1;

    memcpy(full, header, header_len);
    memcpy(full + header_len, data, len);

    ObjectID id;
    compute_hash(full, total_len, &id);

    if (id_out) *id_out = id;

    if (object_exists(&id)) {
        free(full);
        return 0;
    }

    char path[512];
    object_path(&id, path, sizeof(path));

    char dir[512];
    strncpy(dir, path, sizeof(dir));
    char *slash = strrchr(dir, '/');
    if (!slash) {
        free(full);
        return -1;
    }
    *slash = '\0';
    mkdir(dir, 0755);

    char tmp_path[512];
    if (snprintf(tmp_path, sizeof(tmp_path), "%s/tmpXXXXXX", dir) >=
(int)sizeof(tmp_path)) {
```

```
    free(full);
    return -1;
}

  int fd = mkstemp(tmp_path);
  if (fd < 0) {
      free(full);
      return -1;
  }

  if (write(fd, full, total_len) != (ssize_t)total_len) {
      close(fd);
      unlink(tmp_path);
      free(full);
      return -1;
  }

  fsync(fd);
  close(fd);

  if (rename(tmp_path, path) != 0) {
      unlink(tmp_path);
      free(full);
      return -1;
  }

  int dir_fd = open(dir, O_DIRECTORY);
  if (dir_fd >= 0) {
      fsync(dir_fd);
      close(dir_fd);
  }

  free(full);
  return 0;
}

// Read an object from the store.
```

```
//
// Steps:
//   1. Build the file path from the hash using object_path()
//   2. Open and read the entire file
//   3. Parse the header to extract the type string and size
//   4. Verify integrity: recompute the SHA-256 of the file contents
//      and compare to the expected hash (from *id). Return -1 if mismatch.
//   5. Set *type_out to the parsed ObjectType
//   6. Allocate a buffer, copy the data portion (after the \0), set *data_out and *len_out
//
// HINTS - Useful syscalls and functions for this phase:
//   - object_path        : getting the target file path
//   - fopen, fread, fseek: reading the file into memory
//   - memchr             : safely finding the '\0' separating header and data
//   - strncmp            : parsing the type string ("blob", "tree", "commit")
//   - compute_hash       : re-hashing the read data for integrity verification
//   - memcmp             : comparing the computed hash against the requested hash
//   - malloc, memcpy     : allocating and returning the extracted data
//
// The caller is responsible for calling free(*data_out).
// Returns 0 on success, -1 on error (file not found, corrupt, etc.).

// Phase 1 complete: object storage implemented

int object_read(const ObjectID *id, ObjectType *type_out, void **data_out, size_t
*len_out) {
    char path[512];
    object_path(id, path, sizeof(path));

    FILE *fp = fopen(path, "rb");
    if (!fp) return -1;

    fseek(fp, 0, SEEK_END);
    long file_size = ftell(fp);
    rewind(fp);

    if (file_size <= 0) {
```

```
    fclose(fp);
    return -1;
}

char *buffer = malloc(file_size);
if (!buffer) {
    fclose(fp);
    return -1;
}

if (fread(buffer, 1, file_size, fp) != (size_t)file_size) {
    free(buffer);
    fclose(fp);
    return -1;
}
fclose(fp);

ObjectID computed;
compute_hash(buffer, file_size, &computed);

if (memcmp(computed.hash, id->hash, HASH_SIZE) != 0) {
    free(buffer);
    return -1;
}

char *null_pos = memchr(buffer, '\0', file_size);
if (!null_pos) {
    free(buffer);
    return -1;
}

char type_str[16];
size_t size;

if (sscanf(buffer, "%15s %zu", type_str, &size) != 2) {
    free(buffer);
    return -1;
```

```
    }

    if (strcmp(type_str, "blob") == 0) *type_out = OBJ_BLOB;
    else if (strcmp(type_str, "tree") == 0) *type_out = OBJ_TREE;
    else if (strcmp(type_str, "commit") == 0) *type_out = OBJ_COMMIT;
    else {
        free(buffer);
        return -1;
    }

    char *data_start = null_pos + 1;

    *data_out = malloc(size);
    if (!*data_out) {
        free(buffer);
        return -1;
    }

    memcpy(*data_out, data_start, size);
    *len_out = size;

    free(buffer);
    return 0;
}
```

2. **tree.c**

```
// tree.c — Tree object serialization and construction
//
// PROVIDED functions: get_file_mode, tree_parse, tree_serialize
// TODO functions:     tree_from_index
//
// Binary tree format (per entry, concatenated with no separators):
```

```
//   "<mode-as-ascii-octal> <name>\0<32-byte-binary-hash>"

#include "index.h"
#include "tree.h"
#include "pes.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>

int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out);

// ─── Mode Constants ──────────────
```

```
// Parse binary tree data into a Tree struct safely.
int tree_parse(const void *data, size_t len, Tree *tree_out) {
    tree_out->count = 0;
    const uint8_t *ptr = (const uint8_t *)data;
    const uint8_t *end = ptr + len;

    while (ptr < end && tree_out->count < MAX_TREE_ENTRIES) {
        TreeEntry *entry = &tree_out->entries[tree_out->count];

        const uint8_t *space = memchr(ptr, ' ', end - ptr);
        if (!space) return -1;

        char mode_str[16] = {0};
        size_t mode_len = space - ptr;
        if (mode_len >= sizeof(mode_str)) return -1;
        memcpy(mode_str, ptr, mode_len);
        entry->mode = strtol(mode_str, NULL, 8);

        ptr = space + 1;

        const uint8_t *null_byte = memchr(ptr, '\0', end - ptr);
        if (!null_byte) return -1;

        size_t name_len = null_byte - ptr;
        if (name_len >= sizeof(entry->name)) return -1;
        memcpy(entry->name, ptr, name_len);
        entry->name[name_len] = '\0';

        ptr = null_byte + 1;

        if (ptr + HASH_SIZE > end) return -1;
        memcpy(entry->hash.hash, ptr, HASH_SIZE);
        ptr += HASH_SIZE;

        tree_out->count++;
```

```
    }
    return 0;
}

// Helper for sorting
static int compare_tree_entries(const void *a, const void *b) {
    return strcmp(((const TreeEntry *)a)->name, ((const TreeEntry *)b)->name);
}

// Serialize tree
int tree_serialize(const Tree *tree, void **data_out, size_t *len_out) {
    size_t max_size = tree->count * 296;
    uint8_t *buffer = malloc(max_size);
    if (!buffer) return -1;

    Tree sorted_tree = *tree;
    qsort(sorted_tree.entries, sorted_tree.count, sizeof(TreeEntry),
compare_tree_entries);

    size_t offset = 0;
    for (int i = 0; i < sorted_tree.count; i++) {
        const TreeEntry *entry = &sorted_tree.entries[i];

        int written = sprintf((char *)buffer + offset, "%o %s", entry->mode,
entry->name);
        offset += written + 1;

        memcpy(buffer + offset, entry->hash.hash, HASH_SIZE);
        offset += HASH_SIZE;
    }

    *data_out = buffer;
    *len_out = offset;
    return 0;
}
```

```
// ─── FIXED IMPLEMENTATION ──────────────
```

```
            sub_entries[sub_count].mode = entries[i].mode;
            sub_entries[sub_count].hash = entries[i].hash;

            sub_count++;
            i++;
        }

        ObjectID sub_id;
        write_tree_recursive(sub_entries, sub_count, &sub_id);

        TreeEntry *e = &tree.entries[tree.count++];

        e->mode = MODE_DIR;
        e->hash = sub_id;
        strcpy(e->name, dirname);
    }
}

void *buffer;
size_t size;

if (tree_serialize(&tree, &buffer, &size) != 0) return -1;

if (object_write(OBJ_TREE, buffer, size, id_out) != 0) {
    free(buffer);
    return -1;
}

free(buffer);
return 0;
}

// MAIN FUNCTION
int tree_from_index(ObjectID *id_out) {
    Index index;
```

```
    if (index_load(&index) != 0) return -1;

    // sort index
    for (int i = 0; i < index.count - 1; i++) {
        for (int j = i + 1; j < index.count; j++) {
            if (strcmp(index.entries[i].path,
                    index.entries[j].path) > 0) {

                IndexEntry temp = index.entries[i];
                index.entries[i] = index.entries[j];
                index.entries[j] = temp;
            }
        }
    }

    return write_tree_recursive(index.entries, index.count, id_out); //recursive helper
function
}
```

3. **index.c**

```
// index.c — Staging area implementation
//
// Text format of .pes/index (one entry per line, sorted by path):
//
//   <mode-octal> <64-char-hex-hash> <mtime-seconds> <size> <path>
//
// Example:
//   100644 a1b2c3d4e5f6...  1699900000 42 README.md
//   100644 f7e8d9c0b1a2...  1699900100 128 src/main.c
```

```
//
// This is intentionally a simple text format. No magic numbers, no
// binary parsing. The focus is on the staging area CONCEPT (tracking
// what will go into the next commit) and ATOMIC WRITES (temp+rename).
//
// PROVIDED functions: index_find, index_remove, index_status
// TODO functions:     index_load, index_save, index_add

#include "index.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <dirent.h>
int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out);
// ─── PROVIDED ──────────────
```

```
            int remaining = index->count - i - 1;
            if (remaining > 0)
                memmove(&index->entries[i], &index->entries[i + 1],
                        remaining * sizeof(IndexEntry));
            index->count--;
            return index_save(index);
        }
    }
    fprintf(stderr, "error: '%s' is not in the index\n", path);
    return -1;
}

// Print the status of the working directory.
//
// Identifies files that are staged, unstaged (modified/deleted in working dir),
// and untracked (present in working dir but not in index).
// Returns 0.
int index_status(const Index *index) {
    printf("Staged changes:\n");
    int staged_count = 0;
    // Note: A true Git implementation deeply diffs against the HEAD tree here.
    // For this lab, displaying indexed files represents the staging intent.
    for (int i = 0; i < index->count; i++) {
        printf("  staged:     %s\n", index->entries[i].path);
        staged_count++;
    }
    if (staged_count == 0) printf("  (nothing to show)\n");
    printf("\n");

    printf("Unstaged changes:\n");
    int unstaged_count = 0;
    for (int i = 0; i < index->count; i++) {
        struct stat st;
        if (stat(index->entries[i].path, &st) != 0) {
            printf("  deleted:    %s\n", index->entries[i].path);
```

```
            unstaged_count++;
        } else {
            // Fast diff: check metadata instead of re-hashing file content
            if (st.st_mtime != (time_t)index->entries[i].mtime_sec || st.st_size !=
(off_t)index->entries[i].size) {
                printf("  modified:   %s\n", index->entries[i].path);
                unstaged_count++;
            }
        }
    }
    if (unstaged_count == 0) printf("  (nothing to show)\n");
    printf("\n");

    printf("Untracked files:\n");
    int untracked_count = 0;
    DIR *dir = opendir(".");
    if (dir) {
        struct dirent *ent;
        while ((ent = readdir(dir)) != NULL) {
            // Skip hidden directories, parent directories, and build artifacts
            if (strcmp(ent->d_name, ".") == 0 || strcmp(ent->d_name, "..") == 0)
continue;
            if (strcmp(ent->d_name, ".pes") == 0) continue;
            if (strcmp(ent->d_name, "pes") == 0) continue; // compiled executable
            if (strstr(ent->d_name, ".o") != NULL) continue; // object files

            // Check if file is tracked in the index
            int is_tracked = 0;
            for (int i = 0; i < index->count; i++) {
                if (strcmp(index->entries[i].path, ent->d_name) == 0) {
                    is_tracked = 1;
                    break;
                }
            }
```

```
        if (!is_tracked) {
            struct stat st;
            stat(ent->d_name, &st);
            if (S_ISREG(st.st_mode)) { // Only list regular files for simplicity
                printf("  untracked:  %s\n", ent->d_name);
                untracked_count++;
            }
        }
    }
    closedir(dir);
  }
  if (untracked_count == 0) printf("  (nothing to show)\n");
  printf("\n");

  return 0;
}

// ——— TODO: Implement these ———————————————————————————————————————————————————————————
———

// Load the index from .pes/index.
//
// HINTS - Useful functions:
//   - fopen (with "r"), fscanf, fclose : reading the text file line by line
//   - hex_to_hash                      : converting the parsed string to ObjectID
//
// Returns 0 on success, -1 on error.
int index_load(Index *index) {
  index->count = 0;

  FILE *fp = fopen(INDEX_FILE, "r");
  if (!fp) {
    return 0;
  }
```

```
    while (index->count < MAX_INDEX_ENTRIES) {
        IndexEntry *e = &index->entries[index->count];
        char hex[HASH_HEX_SIZE + 1];

        int rc = fscanf(fp, "%o %64s %lu %u %511[^\n]\n", &e->mode, hex,
&e->mtime_sec, &e->size,
                        e->path);

        if (rc == EOF) break;
        if (rc != 5) {
            fclose(fp);
            return -1;
        }

        if (hex_to_hash(hex, &e->hash) != 0) {
            fclose(fp);
            return -1;
        }

        index->count++;
    }

    fclose(fp);
    return 0;
}

// Save the index to .pes/index atomically.
//
// HINTS - Useful functions and syscalls:
//   - qsort                          : sorting the entries array by path
//   - fopen (with "w"), fprintf      : writing to the temporary file
//   - hash_to_hex                    : converting ObjectID for text output
//   - fflush, fileno, fsync, fclose  : flushing userspace buffers and syncing to disk
//   - rename                         : atomically moving the temp file over the old index
```

```
//
// Returns 0 on success, -1 on error.
static int cmp_entries(const void *a, const void *b) {
    const IndexEntry *ea = (const IndexEntry *)a;
    const IndexEntry *eb = (const IndexEntry *)b;
    return strcmp(ea->path, eb->path);
}

// writes the in-memory staging area
int index_save(const Index *index) {
    qsort((void *)index->entries, index->count, sizeof(IndexEntry), cmp_entries);

    FILE *fp = fopen(".pes/index.tmp", "w");
    if (!fp) return -1;

    for (int i = 0; i < index->count; i++) {
        char hex[HASH_HEX_SIZE + 1];
        hash_to_hex(&index->entries[i].hash, hex);

        fprintf(fp, "%o %s %llu %u %s\n",
                index->entries[i].mode,
                hex,
                (unsigned long long)index->entries[i].mtime_sec,
                index->entries[i].size,
                index->entries[i].path);
    }

    fflush(fp);
    fsync(fileno(fp));
    fclose(fp);

    if (rename(".pes/index.tmp", INDEX_FILE) != 0)
        return -1;

    return 0;
```

```
}

// Stage a file for the next commit.
//
// HINTS - Useful functions and syscalls:
//   - fopen, fread, fclose            : reading the target file's contents
//   - object_write                    : saving the contents as OBJ_BLOB
//   - stat / lstat                    : getting file metadata (size, mtime, mode)
//   - index_find                      : checking if the file is already staged
//
// Returns 0 on success, -1 on error.
int index_add(Index *index, const char *path) {
    struct stat st;
    if (stat(path, &st) != 0) return -1;

    FILE *fp = fopen(path, "rb");
    if (!fp) return -1;

    size_t size = (size_t)st.st_size;
    char *buf = malloc(size ? size : 1);
    if (!buf) {
        fclose(fp);
        return -1;
    }

    if (size > 0 && fread(buf, 1, size, fp) != size) {
        free(buf);
        fclose(fp);
        return -1;
    }
    fclose(fp);

    ObjectID id;
    if (object_write(OBJ_BLOB, buf, size, &id) != 0) {
        free(buf);
```

```
        return -1;
    }
    free(buf);

    IndexEntry *e = index_find(index, path);
    if (!e) {
        if (index->count >= MAX_INDEX_ENTRIES) return -1;
        e = &index->entries[index->count++];
    }

    e->mode = (st.st_mode & S_IXUSR) ? 0100755 : 0100644;
    e->hash = id;
    e->mtime_sec = (uint64_t)st.st_mtime;
    e->size = (uint32_t)st.st_size;

    strncpy(e->path, path, sizeof(e->path) - 1);
    e->path[sizeof(e->path) - 1] = '\0';

    return index_save(index);
}
```

4. **commit.c**

```
// commit.c — Commit creation and history traversal
//
// Commit object format (stored as text, one field per line):
//
//   tree <64-char-hex-hash>
//   parent <64-char-hex-hash>        ← omitted for the first commit
//   author <name> <unix-timestamp>
//   committer <name> <unix-timestamp>
//
//   <commit message>
```

```
//
// Note: there is a blank line between the headers and the message.
//
// PROVIDED functions: commit_parse, commit_serialize, commit_walk,
head_read, head_update
// TODO functions:     commit_create

#include "commit.h"
#include "index.h"
#include "tree.h"
#include "pes.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <inttypes.h>
#include <time.h>
#include <unistd.h>
#include <fcntl.h>

// Declarations
int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out);
int object_read(const ObjectID *id, ObjectType *type_out, void **data_out, size_t
*len_out);

// ─── PROVIDED ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────

// Parse raw commit data into a Commit struct.
int commit_parse(const void *data, size_t len, Commit *commit_out) {
    (void)len;
    const char *p = (const char *)data;
    char hex[HASH_HEX_SIZE + 1];

    // "tree <hex>\n"
```

```
    if (sscanf(p, "tree %64s\n", hex) != 1) return -1;
    if (hex_to_hash(hex, &commit_out->tree) != 0) return -1;
    p = strchr(p, '\n') + 1;

    // optional "parent <hex>\n"
    if (strncmp(p, "parent ", 7) == 0) {
        if (sscanf(p, "parent %64s\n", hex) != 1) return -1;
        if (hex_to_hash(hex, &commit_out->parent) != 0) return -1;
        commit_out->has_parent = 1;
        p = strchr(p, '\n') + 1;
    } else {
        commit_out->has_parent = 0;
    }

    // "author <name> <timestamp>\n"
    char author_buf[256];
    uint64_t ts;
    if (sscanf(p, "author %255[^\n]\n", author_buf) != 1) return -1;
    // split off trailing timestamp
    char *last_space = strrchr(author_buf, ' ');
    if (!last_space) return -1;
    ts = (uint64_t)strtoull(last_space + 1, NULL, 10);
    *last_space = '\0';
    snprintf(commit_out->author, sizeof(commit_out->author), "%s", author_buf);
    commit_out->timestamp = ts;
    p = strchr(p, '\n') + 1;  // skip author line
    p = strchr(p, '\n') + 1;  // skip committer line
    p = strchr(p, '\n') + 1;  // skip blank line

    snprintf(commit_out->message, sizeof(commit_out->message), "%s", p);
    return 0;
}

// Serialize a Commit struct to the text format.
// Caller must free(*data_out).
```

```
int commit_serialize(const Commit *commit, void **data_out, size_t *len_out) {
    char tree_hex[HASH_HEX_SIZE + 1];
    char parent_hex[HASH_HEX_SIZE + 1];
    hash_to_hex(&commit->tree, tree_hex);

    char buf[8192];
    int n = 0;
    n += snprintf(buf + n, sizeof(buf) - n, "tree %s\n", tree_hex);
    if (commit->has_parent) {
        hash_to_hex(&commit->parent, parent_hex);
        n += snprintf(buf + n, sizeof(buf) - n, "parent %s\n", parent_hex);
    }
    n += snprintf(buf + n, sizeof(buf) - n,
                  "author %s %" PRIu64 "\n"
                  "committer %s %" PRIu64 "\n"
                  "\n"
                  "%s",
                  commit->author, commit->timestamp,
                  commit->author, commit->timestamp,
                  commit->message);

    *data_out = malloc(n + 1);
    if (!*data_out) return -1;
    memcpy(*data_out, buf, n + 1);
    *len_out = (size_t)n;
    return 0;
}

// Walk commit history from HEAD to the root.
int commit_walk(commit_walk_fn callback, void *ctx) {
    ObjectID id;
    if (head_read(&id) != 0) return -1;

    while (1) {
        ObjectType type;
```

```
        void *raw;
        size_t raw_len;
        if (object_read(&id, &type, &raw, &raw_len) != 0) return -1;

        Commit c;
        int rc = commit_parse(raw, raw_len, &c);
        free(raw);
        if (rc != 0) return -1;

        callback(&id, &c, ctx);

        if (!c.has_parent) break;
        id = c.parent;
    }
    return 0;
}

// Read the current HEAD commit hash.
int head_read(ObjectID *id_out) {
    FILE *f = fopen(HEAD_FILE, "r");
    if (!f) return -1;
    char line[512];
    if (!fgets(line, sizeof(line), f)) { fclose(f); return -1; }
    fclose(f);
    line[strcspn(line, "\r\n")] = '\0'; // strip newline

    char ref_path[512];
    if (strncmp(line, "ref: ", 5) == 0) {
        snprintf(ref_path, sizeof(ref_path), "%s/%s", PES_DIR, line + 5);
        f = fopen(ref_path, "r");
        if (!f) return -1; // Branch exists but has no commits yet
        if (!fgets(line, sizeof(line), f)) { fclose(f); return -1; }
        fclose(f);
        line[strcspn(line, "\r\n")] = '\0';
    }
```

```
    return hex_to_hash(line, id_out);
}

// Update the current branch ref to point to a new commit atomically.
int head_update(const ObjectID *new_commit) {
    FILE *f = fopen(HEAD_FILE, "r");
    if (!f) return -1;
    char line[512];
    if (!fgets(line, sizeof(line), f)) { fclose(f); return -1; }
    fclose(f);
    line[strcspn(line, "\r\n")] = '\0';

    char target_path[520];
    if (strncmp(line, "ref: ", 5) == 0) {
        snprintf(target_path, sizeof(target_path), "%s/%s", PES_DIR, line + 5);
    } else {
        snprintf(target_path, sizeof(target_path), "%s", HEAD_FILE); // Detached
HEAD
    }

    char tmp_path[528];
    snprintf(tmp_path, sizeof(tmp_path), "%s.tmp", target_path);

    f = fopen(tmp_path, "w");
    if (!f) return -1;

    char hex[HASH_HEX_SIZE + 1];
    hash_to_hex(new_commit, hex);
    fprintf(f, "%s\n", hex);

    fflush(f);
    fsync(fileno(f));
    fclose(f);

    return rename(tmp_path, target_path);
```

```
}

// ─── TODO: Implement these ──────────────────────────────────────────────────────────────────────────────────────

// Create a new commit from the current staging area.
//
// HINTS - Useful functions to call:
//   - tree_from_index   : writes the directory tree and gets the root hash
//   - head_read         : gets the parent commit hash (if any)
//   - pes_author        : retrieves the author name string (from pes.h)
//   - time(NULL)        : gets the current unix timestamp
//   - commit_serialize  : converts the filled Commit struct to a text buffer
//   - object_write      : saves the serialized text as OBJ_COMMIT
//   - head_update       : moves the branch pointer to your new commit
//
// Returns 0 on success, -1 on error.
int commit_create(const char *message, ObjectID *commit_id_out) {
    ObjectID tree_id, parent_id;

    // 1. Building tree from index
    if (tree_from_index(&tree_id) != 0)
        return -1;

    // 2. Checking parent
    int has_parent = (head_read(&parent_id) == 0);

    // 3. Filling commit struct
    Commit commit = {0};
    commit.tree = tree_id;
    commit.has_parent = has_parent;

    if (has_parent)
        commit.parent = parent_id;
```

```
    // 4. Author + timestamp
    const char *author = pes_author();
    snprintf(commit.author, sizeof(commit.author), "%s", author);
    commit.timestamp = (uint64_t)time(NULL);

    // 5. Message
    strncpy(commit.message, message, sizeof(commit.message) - 1);
    commit.message[sizeof(commit.message) - 1] = '\0';

    // 6. Serialize
    void *data;
    size_t len;
    if (commit_serialize(&commit, &data, &len) != 0)
        return -1;

    // 7. Write commit object
    if (object_write(OBJ_COMMIT, data, len, commit_id_out) != 0) {
        free(data);
        return -1;
    }

    free(data);

    // 8. Update HEAD
    if (head_update(commit_id_out) != 0)
        return -1;

    // 9. Print
    char hex[HASH_HEX_SIZE + 1];
    hash_to_hex(commit_id_out, hex);
    printf("Committed as %s\n", hex);

    return 0;
}
```

Analysis Questions (written answers)

Q5.1) A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

pes checkout <branch> updates .pes/HEAD to reference .pes/refs/heads/<branch>, ensuring future commits go to that branch. It then reads the commit hash stored in that branch, loads its tree, and reconstructs the working directory by writing blob contents and removing files not present in the target tree. The index may also need updating to match this state. The operation is complex because it must avoid overwriting uncommitted changes, handle nested directories recursively, and ensure consistency between HEAD, index, and working directory.

Q5.2) When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

To detect a dirty working directory, the system hashes each tracked file and compares it with the hash stored in the index (last committed state). If the hash differs, the file has been modified. It then compares this file with the version in the target branch’s tree. If the file differs from both the index and the target version, switching branches would overwrite local changes. In such cases, checkout must be aborted to prevent data loss.

Q5.3) "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In a detached HEAD state, HEAD stores a commit hash instead of a branch reference. New commits can still be created, but no branch points to them, so they are not part of

any branch history. These commits remain accessible via their hashes but can become unreachable if the user switches away. Users can recover them by checking out the commit hash again or creating a new branch pointing to it. Otherwise, they may eventually be removed by garbage collection

Q6.1) Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Garbage collection can be implemented using a mark-and-sweep algorithm. In the mark phase, start from all branch heads in .pes/refs/heads/ and traverse commits, their parent commits, associated trees, and blobs, marking each visited object. A hash set is used to efficiently track reachable object hashes. In the sweep phase, all objects in .pes/objects not in this set are deleted. For a repository with ~100,000 commits and 50 branches, traversal may involve visiting a few hundred thousand objects due to shared history.

Q6.2) Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Running GC concurrently with a commit is unsafe due to race conditions. During a commit, new blobs, trees, and the commit object are written before updating the branch reference. If GC runs in between, it may not see these objects as reachable and delete them. Once the commit finishes, it would reference missing objects, corrupting the repository. Real Git avoids this using atomic updates, object protection mechanisms, reflogs, and locking to prevent GC from interfering with active operations.